PROGRAMMING ASSIGNMENT #3 **Due date: 28.12.2025 23.59**

This assignment requires the design and implementation of a custom Instruction Set Architecture (ISA), which must be created by customizing or updating the existing MIPS-lite single-cycle implementation previously discussed in lab sessions. Development and testing of Verilog code must be performed using the ModelSim simulator. The assignment is structured in two parts: the first involves integrating new instructions into the ISA, and the second focuses on implementing new features within the processor architecture. The processor must be designed as a 32-bit machine, fully capable of processing 32-bit instructions. The register file must contain 16 registers (of reg type in Verilog), each having a 32-bit width. Conversely, both the instruction memory and the data memory should be implemented using 64 memory locations (of reg type in Verilog), each 8-bits wide. The register file must be logically partitioned into specific functional roles, including temporary, saved, stack pointer (sp), and return address (ra) registers, as outlined in Table 3. The integration of the new instructions and features in Part 1 and Part 2 is permitted to involve the addition of new hardware modules to the existing processor architecture in the lab sessions. However, any new module must be accompanied by a figure that illustrates the updated processor block diagram, clearly detailing the module's placement and its specific connections to the rest of the data path and control unit.

Part 1: Implementing New Instructions

This section requires the integration of new instructions into the ISA. The instruction set is provided for reference in Table 1; however, the order of these instructions may be rearranged as necessary, as the table serves purely as an example. For R-type instructions, you have the flexibility to modify the function field (function bits) and the ALU control lines to meet specific implementation requirements, or they may choose to retain their existing definitions.

Additionally, **note that the opcode for each instruction will vary based on your student ID.** The least significant 3 bits of your ID determine the opcode for the first instruction. For example:
Student id: 230062100 -> 100 is the first instruction's opcode.

Table 1. An example of instruction set architecture

opcode (decimal)	opcode (binary)	function field (decimal)	function field (binary)	instruction
100	01100100	1	000001	sll (R-type)*
100	01100100	2	000010	$move$ (R-type)*
100	01100100	3	000011	$nand$ (R-type)*
100	01100100	4	000100	or (R-type)
100	01100100	5	000101	add (R-type)
101	01100101	x	xxxxxx	lw (I-type)
102	01100110	x	xxxxxx	sw (I-type)
103	01100111	x	xxxxxx	beq (I-type)
104	01101000	x	xxxxxx	blt (I-type)*
105	01101001	x	xxxxxx	$subi$ (I-type)*
106	01101010	x	xxxxxx	$addi$ (I-type)*
107	01101011	x	xxxxxx	$beqi$ (B-I-type)*
108	01101100	x	xxxxxx	j (J-type)

There is a total of 13 instructions in Table 1, some of which have been implemented during the lab sessions. Among these, the instructions marked with * are required to be implemented. Below, the instructions that need to be implemented are explained in detail.

1) Register Type (R-type)

8 bits	4 bits	4 bits	4 bits	6 bits	6 bits
opcode[31:24]	rs[23:20]	rt[19:16]	rd[15:12]	shamt[11:6]	funct[5:0]

a) Shift Left Logical (sll): This instruction shifts the value in the rs register to the left by the number of positions specified in the shamt field of the R-type instruction. The shifted result is then written to the rd register. The contents of the rt is ignored.

b) $move$: The instruction moves the value of rs register to the rd register. The contents of rt and shamt fields are typically ignored. Example MIPS code of $move$: $move\ \$rd, \$rs$

c) NAND ($nand$): This instruction performs the bitwise NAND (NOT AND) operation on the values stored in the rs and rt registers. The result of the operation is then written to the rd register.

2) Immediate Type (I-type)

8 bits	4 bits	4 bits	16 bits
opcode[31:24]	rs[23:20]	rt[19:16]	address[15:0]

a) Subtract Immediate ($subi$): The instruction subtracts the 16-bit sign-extended immediate value from the contents of the rs register and writes the result to the rt register.

b) Branch Less Than Equal (blt): The instruction first determines if the value in the rs register is less than the value in the rt register. If the condition is true, the program branches to the target address, which is computed by sign-extending the 16-bit offset, multiplying it by 4, and adding the result to the address of the next instruction (PC + 4). Example MIPS code of blt: $blt\ \$rt, \$rs, immediate$

c) Add Immediate ($addi$): The instruction adds the 16-bit sign-extended immediate value to the contents of the rs register and writes the result to the rt register. Example MIPS code of $addi$: $addi\ \$rt, \$rs, immediate$

3) Branch & Immediate Type (B-I-type)

8 bits	4 bits	4 bits	8 bits	8 bits
opcode[31:24]	rs[23:20]	rt[19:16]	immediate[15:8]	address[7:0]

a) Branch Equal Immediate ($beqi$): The instruction compares the content of the rs register with the 8-bit sign-extended immediate value. If the values are equal, the program branches to the target address using PC-relative addressing. The target address is computed by sign-extending the 8-bit offset, multiplying it by 4, and then adding the result to the address of the next instruction (PC + 4). The content of the rt register is ignored during this operation.

Part 2: Processor Feature Implementation (Stack Management)

This section requires the addition of new features and functionality to the processor. Specifically, while the MIPS-lite processor completed in the laboratory did not include explicit discussion of a stack structure, this part requires the implementation of mechanisms necessary to properly execute instructions that utilize the Stack Pointer (sp).

Stack Overflow/Underflow Detection Requirement

The core feature to be added is robust error checking for the Stack Pointer. Although the current processor might execute stack-related instructions (like $addi$, lw, sw) without additional functionality, it currently provides no warning if the Stack Pointer exceeds its defined limits. The required design change is to integrate the necessary modules or connections into the processor to monitor Stack Pointer limit violations and detect overflow or underflow errors. If an overflow or underflow condition is detected during the execution of critical instructions ($addi$, lw, or sw), the processor must be designed to prevent the execution of that instruction. Furthermore, you are required to add new signal lines or wires for overflow and underflow detection, which activate when the Stack Pointer exceeds its limit.

Roles and Initialization of Register File

To support the desired functionality, the processor's registers must be assigned specific roles, as outlined in Table 2. You are responsible for defining this register mapping based on the functional description provided below. Furthermore, the initial values for the registers within the register file, which are also detailed in Table 2, must be specified.

Table 2. The values and roles of the register file of the processor

Register Index (32b bits)	Assigned Role	Symbol	Value
R0	Zero Register (Contains the value 0)	$\$0$	0
R1	Saved Register	$\$s0$	2
R2	Saved Register	$\$s1$	3
R3	Saved Register	$\$s2$	4
R4	Saved Register	$\$s3$	2
R5	Saved Register	$\$s4$	3
R6	Temporary Registers	$\$t0$	6
R7	Temporary Registers	$\$t1$	9
R8	Temporary Registers	$\$t2$	4
R9	Argument Register	$\$a0$	0
R10	Argument Register	$\$a1$	0
R11	Value Register	$\$v0$	0
R12	Stack Pointer Base Address	$\$spba$	40
R13	Stack Pointer Limit	$\$spl$	2
R14	Stack Pointer	$\$sp$	0
R15	Return Address	$\$ra$	-

Stack Pointer Initialization and Operation

The Stack Pointer must be initialized and operate according to the following rules:

- Empty Stack: When the stack is empty, the Stack Pointer must hold the value 0.
- Push Operation: When pushing (storing) one data item, the Stack Pointer value must decrease to -4.
- Pop Operation: To pop one data item, the Stack Pointer value must be incremented by 4.
- Limit and Base Values:
    - The Stack Pointer Limit (spl) must be initialized to a value of 2. This requires that the stack capacity is limited to storing exactly two words (32-bit units).
    - The Stack Pointer Base Address ($spba$) must be initialized to the value 40 (or 0x28). This configuration means that the stack capacity of two words (32-bit units) is mapped to the data memory, occupying the address range from 32 to 39 (or 0x20 to 0x27), assuming the stack grows downwards.

Data Memory & Stack Allocation

To execute the instruction blocks specified below, the necessary initial values for the processor's data memory locations are presented in Table 3.

Table 3. The values of the data memory

Location Index (8x4= 32b bits)	Value
R0 - R3	1
R4 - R7	2
R8 - R11	2
R12 - R15	5
R16 - R19	3
R20 - R23	3
R24 - R27	56
R28 - R31	5
R32 - R35	34
R36 - R39	0
R40 - R43	0
R44 - R47	0
R48 - R51	0
R52 - R55	0
R56 - R59	0
R60 - R63	0

The physical location of the stack within the data memory is defined by Figure 1. The stack memory space is specifically allocated to the address range spanning from location 32 to location 39 of the data memory. Since the system utilizes 8-bit memory locations (of reg type in Verilog) but processes 32-bit words, this 8-byte address range effectively reserves eight 8-bit memory locations for the stack. Furthermore, the Stack Pointer Limit (spl) is set to 2, which corresponds to the size of $2\ x\ 4 = 8$ locations. All data storage operations involving the stack must be confined within this specific address range.

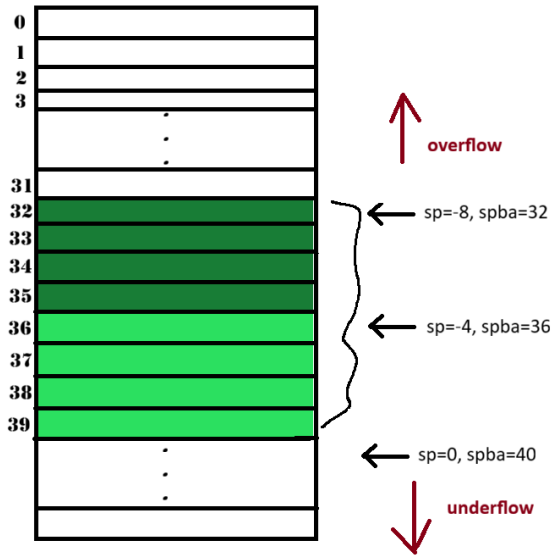


Figure 1. Stack allocation within the data memory

1. Design and Implementation Task

The primary objective is to design **revised single-cycle data path and control units** to create an updated processor. This new design must execute **all previously implemented instructions** as well as the following mandatory new instructions: ***sll, move***, ***nand***, ***blt***, ***subi***, ***addi***, and ***beqi***. Crucially, you must also design and implement the necessary modules and connections to support **Stack Pointer error handling** (i.e., detecting and preventing overflow/underflow errors). After the revised design is complete, the entire updated processor must be implemented using **Verilog**.

2. Deliverables and Reporting

Your final submission must contain the following elements:

- Your complete **Verilog code** and **.dat files** for the register file, data memory, and instruction memory must be submitted. The initial values within the .dat files should be populated as follows: the register file using Table 3, and the data memory using Table 2. The instruction memory .dat file must be filled with machine code derived from your Instruction Set Architecture (ISA) table.

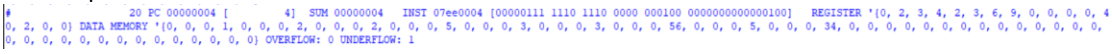
- A **report in pdf format** that includes:
  - Your complete **Instruction Set Architecture (ISA)** table.
  - **A figure illustrating the updated block diagram** and their connections must be included in the report. Please ensure the updated block diagram is **clearly legible**, even if it is drawn by hand.

3. Demonstration and Evaluation Requirements

You will be required to demonstrate your work on a date to be announced later. Successful completion requires the following:

- **Instruction Execution:** During the demonstration, a total of **five specific instruction blocks** will be executed. Ensure **all** implemented instructions are functioning correctly before the demonstration date.

- **Display Requirements:** The ModelSim display must clearly show the contents of the **registers**, **data memory**, and the **currently executing instruction** in a manner similar to that used in the lab sessions. This includes displaying and highlighting the decoded fields of the instruction, such as **opcode**, **rs**, **rt**, **rd**, **address**, and **function field**. In addition, the wire or signal for **overflow** and **underflow** should also be displayed at the completion of **each instruction execution**. If the display requirements are not met, points will be deducted. An example output should be as follows:



- **Initialization:** You may use either **.dat files** or a **testbench** to initialize the memories and registers. If the files required to fill the data memory, register file, and instruction memory (such as .dat files) are not provided, points will be deducted.
  - The **instruction memory** must be initialized with the machine code derived from your designed **Instruction Set Architecture (ISA)**, following a format similar to that presented in Table 1.
  - The **register memory** must be initialized using the values provided in **Table 2**.
  - The **data memory** must be initialized using the values provided in **Table 3**.

- **Simulation Guidance:** During the ModelSim simulation, if an expected instruction does not complete its execution, you may need to increase the **run length** (e.g., to 500 clock cycles) to allow sufficient time for completion. You need to also include the line $\#500\ \$finish;$ in the processor code.

Instruction Blocks (IB) for Demonstration

IB1:	**14pts**	
Adress	**Instruction**	
0	$START$:	$lw\ \$s2, 0(\$t2)$
4		$add\ \$s4, \$s2, \$s0$
8		$beqi\ \$s2, 4, EXIT$
12		$sw\ \$s4, 8(\$0)$
16		$blt\ \$s4, \$s2, NEXT_LOAD$
20		$j\ \ EXIT$
24	$NEXT_LOAD$:	$addi\ \$t2, \$t2, 4$
28		$j\ START$
32	$EXIT$:	$beq\ \$t2, \$t2, 0$

IB2:	**17pts**
Address	**Instruction**
0	$addi\ \$s1, \$s0, 10$
4	$sll\ \$s2, \$s3, 2$
8	$nand\ \$s4, \$s2, \$s3$
12	$subi\ \$s1, \$s1, 1$

IB3: 14pts

Address	Instruction
0	$blt\ \$0, \$s2, EXIT$
4	$move\ \$s0, \$s2$
8	$or\ \$s0, \$s0, \$s3$
12	$EXIT:\ subi\ \$s1, \$s1, 2$

IB4: 18pts

Address	Instruction
0	$addi\ \$sp, \$sp, -4$
4	$sw\ \$t0, 0(\$sp)$
8	$addi\ \$sp, \$sp, -4$
12	$sw\ \$t1, 0(\$sp)$
16	$lw\ \$sp, -4(\$sp)$
20	$sw\ \$t2, 0(\$sp)$
24	$beq\ \$t0, \$t0, 0$

IB5: 17pts

Address	Instruction
0	$START: lw\ \$t2, 0(\$sp)$
4	$addi\ \$sp, \$sp, 4$
8	$lw\ \$t1, 0(\$sp)$
12	$lw\ \$sp, 4(\$sp)$
16	$lw\ \$t0, 0(\$sp)$
20	$beqi\ \$t1, 1, START$
24	$sll\ \$s3, \$s2, 2$

Summary of Requirements:

- You must submit the complete **source code** (including all .v files, .dat initialization files, etc.) along with **a report**.
- This is an **individual assignment**; group work is strictly prohibited. **File sharing between students is unacceptable.**
- **No late submissions** will be accepted.
- The structure of the final submission folder must be as detailed below:

```
├— your_report.pdf
├— coding/ |
        ├— (All Verilog .v files) |
        ├— instruction_memory1.dat |
        ├— instruction_memory2.dat |
        ├— register_file.dat |
        └— data_memory.dat
└— (Other Files)
```

Submission: You are required to submit all your files to Microsoft Teams. Please create a compressed file including all source files and report, and name it as yournamesurname_yourstudentnumber_P3.zip (e.g. If your name and your student number are Name Surname and 202112345678, the file name must be name_surname_202112345678_P3.zip). Your report should only include your processor design and your ISA as provided in Table 1.